Per strutturare il progetto basato sulla **Traccia A (Labirinto)**, ti consiglio di seguire un approccio modulare che separi nettamente la logica del server, la gestione del client e il protocollo di comunicazione.

Ecco una proposta di strutturazione, road map e divisione dei compiti per un team di due persone, basata sui materiali del corso.

1. Architettura del Progetto

Ti suggerisco di organizzare il codice in tre cartelle principali per mantenere l'isolamento dei componenti, come visto negli esempi di Docker:

- **`/common`**: File header (`.h`) contenenti le strutture dati condivise (es. la struttura `Cella`, il `Messaggio` del protocollo) e le costanti (es. dimensioni della mappa, tipi di celle).
- **`/server`**: Logica di generazione del labirinto, gestione della concorrenza e logging delle attività in un file.
- **`/client`**: Interfaccia utente (terminale), gestione dell'input (movimenti) e visualizzazione delle mappe locale e globale.

2. Road Map Consigliata

Fase 1: Protocollo e Connessione (Sincrona)

- Definire il protocollo applicativo (es. tipi di messaggi per movimento, aggiornamento mappa, errori).
- Realizzare un server base e un client che si connettono tramite socket TCP e si scambiano un "Hello" con il nickname.

Fase 2: Motore del Gioco (Server)

- Implementare la generazione randomica del labirinto (matrice 2D) con muri, uscite e oggetti.
- Gestire la posizione del giocatore e la logica di movimento (validazione spostamento, raccolta oggetti).

Fase 3: Concorrenza e Visualizzazione (Core del Progetto)

- Rendere il server **concorrente** utilizzando i **Thread** (`pthread_create`) o il multiplexing con `select()` per gestire più client simultaneamente.
- Implementare la "mascheratura" della mappa: il server deve tenere traccia delle celle scoperte per ogni singolo utente.
- Aggiungere l'invio periodico della mappa globale ogni T secondi, possibilmente usando un thread dedicato o segnali di allarme.

Fase 4: Logging e Raffinatezze

- Implementare il sistema di **logging** su file nel server (data/ora connessioni, conquiste).
- Gestire la terminazione pulita del server e la disconnessione dei client tramite segnali (es. SIGINT).
- (Opzionale) Dockerizzare il sistema per facilitare i test di rete tra container diversi.

3. Divisione delle Responsabilità (2 Persone)

Per un lavoro equilibrato, potete dividervi tra chi si occupa del "motore" e chi dell' "interfaccia e comunicazione":

Persona A: Lo Sviluppatore Server (Backend)

- **Gestione Labirinto**: Codice per generare la matrice e gestire la logica fisica (muri, oggetti).
- **Concorrenza**: Implementazione del server multithreaded o con `select()`.
- **Logging**: Gestione della scrittura asincrona su file dei log delle attività.
- **Sincronizzazione**: Gestione di mutex e variabili di condizione per evitare race condition sui dati condivisi del labirinto.

Persona B: Lo Sviluppatore Client & Protocollo (Frontend/Comms)

- **Interfaccia Client**: Gestione dell'input utente e rendering testuale delle mappe (locale e globale).
- **Protocollo**: Implementazione delle funzioni di encoding/decoding dei messaggi tra socket (es. trasformazione di array in stringhe e viceversa).
- **Comunicazione**: Gestione del socket lato client e multiplexing tramite `select()` per ascoltare contemporaneamente l'utente (tastiera) e il server.
- **Relazione**: Stesura della documentazione tecnica, inclusa la guida d'uso e la descrizione del protocollo applicativo.

Consigli Tecnici dai Documenti

- **Server Concorrente**: Se usate i thread, ricordate che ogni client dovrebbe avere un thread dedicato che gestisce il suo ciclo di vita.
- **Standard I/O**: Il server non deve usare `stdout` per i log, ma scrivere direttamente su un file descrittore aperto con `open()` e `O_APPEND`.
- **Persistenza**: Usate i **Volumi Docker** (Bind Mounts) se volete che i file di log del server siano facilmente accessibili sulla macchina host durante i test.